Template

Primary Actor:
Stakeholders and interests:
Preconditions:
Success guarantee:
Main success scenario:
Extensions:
Special requirements:
Technology and data variation list:
Frequency of occurrence:

Aaron - Routing/Navigation

- **Use Case 1A:** Group individuals into routes
  - **Primary Actor:** Driver
  - **Stakeholders and interests:**
    Driver: Wants to be able to efficiently get to each person they need to pick up
    Passenger: Wants to get to their designated location quickly
  - **Preconditions:**
    Driver is an authorized member of the group, and all passengers are also a member of the same group
  - **Success guarantee:**
    The driver is given a route that they can follow to go to the locations of each passenger and arrive at the final destination.
  - **Main success scenario:**
    1. Each passenger confirms that they are ready to be picked up and their pickup location.
    2. The driver is assigned a group of passengers.
    3. System generates route between passenger locations and final destination using an external Map system.
  - **Extensions:**
    3a. External map system doesn't respond
    1. System displays an error and asks the driver to try again soon.
    2. System displays passenger locations and contact information, so the driver can proceed in their own way if they desire.
  - **Special requirements:**

Route is reasonably optimized so that getting to all passengers doesn't take too much time.
    - **Technology and data variation list:**
    - **Frequency of occurrence:** Often, main function of the app
- **Use Case 1B:** Max passenger utilization
    - **Primary Actor:** Driver
    - **Stakeholders and interests:**
      Driver: Has limits of how many passengers they can travel with at a time
    - **Preconditions:** Driver is in a group
    - **Success guarantee:** Route generated by system respects the maximum amount of passengers the driver's vehicle can support
    - **Main success scenario:**
        1. Many passengers indicate they are ready to be picked up in the group
        2. The driver determines how many people they are able to transport, and enters that number into the app as their maximum passenger count
        3. The system only assigns that many passengers to the driver to be picked up.
        4. The system continues with generating a route as in Use Case 1A.
    - **Extensions:**
      1a) Some passengers may represent multiple real people that are getting picked up together. In this case, passengers can also indicate how many people they are joining the group with.
    - **Special requirements:**
    - **Technology and data variation list:**
    - **Frequency of occurrence:** Often, occurs whenever a driver is starting to pick passengers up
- Optimize stop order (time/distance)
    - Non-functional requirement
- **Use Case 1C:** Dynamic reroute
    - **Primary Actor:** Driver
    - **Stakeholders and interests:**
      Driver: Get updated whenever a passenger needs to be picked up in a new location
      Passenger: Ensure that if they need to move somewhere else after

confirming their location they can update it in the app

- **Preconditions:**
  A route between a group of passengers has already been created
- **Success guarantee:** A new route has been generated for the driver with the updated passenger locations and they have been notified of the change.
- **Main success scenario:**
  1) A passenger moves to a new location
  2) In the app, the passenger updates their pickup location to their new location
  3) The system updates the driver with the new passenger information
  4) The system generates a new route with the new locations of all the passengers
  5) The app notifies the driver that there is a new route for their assigned passengers.
  6) The driver taps the notification and the system displays the new route
- **Extensions:**
- **Special requirements:**
- **Technology and data variation list:**
  5a) Since the driver is intended to use an external navigation app while driving the route, this would most likely be a high-priority push notification. If the app is open for whatever reason, it can be an in-app notification instead.
- **Frequency of occurrence:** Infrequent

- **Use Case 1D:** Export route
  - **Primary Actor:** Driver
  - **Stakeholders and interests:**
    Driver: Needs to be able to get to the pickup location, but won't always know how to drive the exact routes
  - **Preconditions:** Route has already been generated and displayed to the driver
  - **Success guarantee:** The driver has switched to their navigation app of choice. The navigation app is set to navigate along the route generated by the app.
  - **Main success scenario:**
    1) System displays the generated route to the driver

2) Driver taps the export button in the app
3) System displays a list of options provided by the device to choose from for exporting the route
4) Driver selects their preferred navigation app
5) System shares route with the selected navigation app
6) Device opens navigation app to the exported route

- **Extensions:**
- **Special requirements:**
- **Technology and data variation list:**
  i. 3) The device may only have one option or a default navigation app configured. In this case, the system can directly open the app without asking the driver to select one
- **Frequency of occurrence:** Often, happens every time the driver wants to use a route

Jacob - Queue/Timing

<u>UC: User joins pickup queue</u>
Primary Actor: Rider
Stakeholders and interests: Rider, wants to be added to pickup queue
Preconditions: rider is part of a group and an event
Success guarantee:

- Rider is added to the pickup queue

Main success scenario:

1. App prompts rider to join queue
2. User selects to be added to pickup queue
3. GPS gets rider's location
4. Rider is added to stop
5. Rider is added to queue with stop
6. APP displays success message

Extensions:
3a. GPS fails to get rider's location
4a. App displays error message, and prompts user to try again
Special requirements:

- Rider must be added to queue in under 5 seconds

Technology and data variation list:

- GPS location via mobile phone
- Real-time database updates

Frequency of occurrence:

- high

<u>UC: Edit Pickup Request</u>

Primary Actor: Rider
Stakeholders and interests: Rider, wants flexibility with their pickup
Preconditions: Rider must currently be in queue for pickup
Success guarantee:

- Rider is removed from pickup queue
- Rider's stop is changed

Main success scenario:

1. User is prompted options: "cancel request", "update stop location" by the system
2. User chooses update stop location
3. GPS gets user's location
4. App updates user's pickup location
5. App displays success message

Extensions:

2a. User chooses "cancel request"

3a. User is removed from queue

4a. App displays success message

3b. GPS is unable to get user's location

4b. App displays error message and prompts the user to retry

5b. Repeat 1

Special requirements:

- Routing logic should be updated to increase efficiency

Technology and data variation list:

- GPD via mobile device

Frequency of occurrence: medium

<u>UC: Queue Status Tracking</u>

Primary Actor: Rider
Stakeholders and interests: Rider, wants to know how many people are ahead of them in the queue
Preconditions: Rider must be in a pickup queue
Success guarantee:

- The position of the rider in the pickup queue is displayed

Main success scenario:

1. User selects “view queue status”
2. App checks for queue
3. App finds location of user in queue
4. App displays position of user in the queue

Extensions:

2a. App loses connection and cannot access the queue

3a. App displays error message and prompts user to retry

4a. Repeat 2

3b. User is not found in queue

4b. Display error message

Special requirements:

- Adjust to queue updates

Technology and data variation list:

- network connectivity

Frequency of occurrence: high

<u>UC: Wait time calculation</u>
Primary Actor: Rider
Stakeholders and interests: Rider, wants to know how long they'll have to wait to be picked up
Preconditions: User must currently be in a pickup queue
Success guarantee:

- Wait time for the rider is displayed

Main success scenario:

1. User selects "calculate wait time"
2. App checks time for each planned route before the user's route
3. App displays estimated wait time

Extensions:

2a. App loses connection to server and cannot retrieve queue info

3a. App displays error message and prompts user to try again

4a. Repeat 2
Special requirements:

- Use traffic data to enhance prediction

Technology and data variation list:

- Network connectivity

Frequency of occurrence: high

<u>UC: Calculate ETA per Stop</u>
Primary Actor: Rider
Stakeholders and interests: Rider, wants to know when the driver will get to each stop
Preconditions: user is added to a route
Success guarantee:

- Eta per stop is displayed

Main success scenario:

1. Rider chooses “calculate ETA per stop”
2. App gets driver’s route list
3. App calculates estimated arrival times
4. App displays estimated arrival times

Extensions:

2a. App loses connection and cannot retrieve route data

3a. App displays error message and prompts user to retry

4a. Repeat 2
Special requirements: Use traffic data to enhance prediction
Technology and data variation list: Network connectivity
Frequency of occurrence: medium

Sam - Groups/Events + Admin

- **Group/event creation**
    - **Primary Actor**: Group Creater
    - **Stakeholders and interests:**
        - Creator/Admin: wants a private group to organize friends for rides/events.
        - Members: want a simple way to join the correct group.
        - System: wants unique groups and controlled access.
- **Preconditions**:
    - User has an app identity (logged in or persistent session).
    - System is online.
- **Success guarantee**:
    - A Group is created, the creator becomes Admin, and a join method (access code + QR) is available.
- **Main success scenario**:
    - User taps Create Group.
    - System prompts for group details (name, optional description).
    - User enters details and confirms.
    - System creates the Group and assigns the user as Admin.
    - System generates a Group access code.
    - System generates a QR code representing the join code.
    - System shows the Group page with the sharing options.
- **Extension**:
    - 2a. User cancels → System returns to previous screen; no group created.
    - 3a. Group name missing/invalid → System prompts for correction.
    - 5a. Code generation fails → System creates Group but disables joining; prompts Admin to retry.
- **Special requirements**::
    - Access code must be difficult to guess (randomized, adequate length).
- **Technology and data variation list**:
    - Group description can be blank.
    - QR may encode a deep link or (groupId + code).
- **Frequency of occurrence:**
    - Occasional

- **<u>Create Event in Group</u>**
    - **Primary Actor**: Group Admin
    - **Stakeholders and interests:**
        - Admin/Organizer: wants to schedule an outing and coordinate participants.
        - Members: want clear time/location and easy joining.
        - System: wants events correctly linked to groups.
- **Preconditions**:
        - Admin is a member of the Group.
        - Admin has permission to create events.
- **Success guarantee**:
    - An Event is created under the Group with join enabled (access code + QR).
- **Main success scenario**:
    - Admin opens the Group.
    - Admin taps Create Event.
    - System prompts for event details (title, date/time, optional meetup location).
    - Admin enters details and confirms.
    - System creates the Event linked to the Group.
    - System generates an Event access code and Event QR code.
    - System displays the Event page with share options.
- **Extension**:
    - 3a. Invalid date/time (past) → System requests correction.
    - 6a. Code/QR generation fails → Event exists but joining disabled; Admin prompted to retry.
- **Special requirements**::
    - Each Event must belong to exactly one Group.
- **Technology and data variation list**:
    - Location can be none / typed / map pin.
    - Codes can be event-specific or group-reused (design choice).
- **Frequency of occurrence:**
    - Medium

- ## Join Group/Event via Access Code or QR
    - **Primary Actor**: Invited User
    - **Stakeholders and interests:**
        - Invited user: wants quick joining.
        - Admin: wants only intended people to join.
        - System: wants to prevent unauthorized access.
- **Preconditions**:
    - Group/Event exists and joining is enabled.
    - User is not banned/blocked.
- **Success guarantee**:
    - User becomes a member/participant and gains access.
- **Main success scenario**:
    - User taps Join.
    - System offers Enter Code or Scan QR.
    - User enters the code or scans the QR.
    - System validates the code and checks join rules.
    - System adds the user to the Group/Event membership list.
    - System confirms success and opens the Group/Event page.
- **Extension**:
    - 3a. Camera permission denied/scan fails → System offers manual code entry.
    - 4a. Code invalid/expired → System shows error and allows retry.
    - 4b. Joining disabled → System denies and tells user to contact Admin.
    - 4c. Too many failed attempts → System rate-limits attempts temporarily.
- **Special requirements**::
    - Rate-limit failed join attempts.
- **Technology and data variation list**:
    - Code entry: case-insensitive recommended.
    - QR may be deep link or raw token
- **Frequency of occurrence:**
    - Often

- ## Admin Manage Access Codes (Rotate / Enable / Disable)

    - **Primary Actor**: Admin
    - **Stakeholders and interests:**
        - Admin: wants to revoke leaked codes and control access.
        - Members: want privacy and consistent access.
        - System: wants consistent enforcement.
- **Preconditions**:
    - Actor is Admin of the Group/Event.
    - Group/Event exists
- **Success guarantee**:
    - Code/join settings are updated and enforced (old code invalid after rotation; joining blocked if disabled).
- **Main success scenario**:
    - Admin opens Group/Event Settings.
    - Admin selects Access Code Settings.
    - System displays join status and code options.
    - Admin chooses Rotate Code, Disable Joining, or Enable Joining.
    - System applies the update (generate new code / toggle joining).
    - System confirms the change.
- **Extension**:
    - 5a. Network/server error → System shows failure and keeps prior settings.
    - 4a. Rotate code → System invalidates the old code immediately (or after grace period if supported).
- **Special requirements**::
    - Code changes must take effect quickly.
- **Technology and data variation list**:
    - Code display may be masked until “Reveal”.
    - Group vs event codes (design choice).
- **Frequency of occurrence:**
    - Infrequent

- **<u>Admin Remove User from Group/Event</u>**
    - **Primary Actor**: Admin
    - **Stakeholders and interests:**
        - Admin: wants to remove disruptive/unintended users.

        - Removed user: wants clarity that access is revoked.
        - System: wants consistent membership rules
- **Preconditions**:
    - Actor is Admin.
    - Target user is currently a member/participant.
- **Success guarantee**:
    - Target user is removed and loses access.
- **Main success scenario**:
    - Admin opens the member list.
    - Admin selects a user.
    - System shows user details and admin actions.
    - Admin taps Remove and confirms.
    - System removes the user from membership records.
    - System blocks the user from accessing the Group/Event going forward.
- **Extension**:
    - 4a. Admin cancels → No change.
    - 4b. Admin tries to remove the only Admin (self) → System blocks unless another admin exists/assigned.
- **Special requirements**::
    - Prevent "orphan" groups/events with zero admins (if your design requires an admin).
- **Technology and data variation list**:
    - Remove from event only vs remove from group.
    - Optional ban flag.
- **Frequency of occurrence:**
    - Rare

Aanya - Messaging

- DD and Rider messaging
    - **Primary Actor:**
        - Passenger or Driver
    - **Stakeholders and interests:**
        - Passenger: wants to communicate with their assigned drive about pickup details, delays, or location changes with reliability and quickness
        - Driver: wants to coordinate with passengers about arrival time,

pickup location, or other logistics without being distracted while driving

- **Preconditions:**
  - Both passenger and driver are members of the same group
  - Driver has been assigned to pick up the passenger
  - Both are users who are logged into the app on it's latest version (if new changes are made)
- **Success guarantee:**
  - Messages are successfully sent and received between drivers and passengers
  - Message history is preserved for the duration of the event within their group
  - Both parties can view conversation history
- **Main success scenario:**
  1. Passenger opens app and navigated to their assigned drivers profile
  2. Passenger selects "message driver" option
  3. Passenger types a message
  4. System sends message to driver
  5. Driver received notification of a new message
  6. Driver opens message to read
  7. Driver replies to message
  8. System delivers reply to passenger
  9. Passenger opens message to read
- **Extensions:**
  - 4a: network connection lost
    1. System queues message locally
    2. System automatically sends message once connection is restored
    3. User sees "sending..." status
  - 5a: driver is currently driving
    1. System sends push notification of new message
    2. Driver views message when safe
  - 3a: message contains inappropriate content
    1. System flags message for review
    2. Message is sent but logged to admin
    3. Message is not allowed to be sent due to content
- **Special requirements:**

        - Message history is kept for at least 24 hours after event is over or until event is over
        - Interface must support one-handed messaging for drivers safety
    - **Technology and data variation list:**
        - Push notifications
        - Real time messaging
        - Message encryption for privacy
    - **Frequency of occurrence:**
        - Very often due to primary communication method during active pickups
- Whole group / event chat
    - **Primary Actor:**
        - Group member (any user in the group)
    - **Stakeholders and interests:**
        - Group members: want to coordinate plans, share updates and communicate with group effectively
        - Group admin: wants to broadcast important information for all members
        - Drivers: want to announce when they start pickups or general updates
    - **Preconditions:**
        - User is a member of an active group
        - Group has at least 2 members
        - User is logged into apps latest version
    - **Success guarantee:**
        - Message is delivered to all group members
        - All members can see message history is chronological order
        - Members can identify who sent each message with timestamp
    - **Main success scenario:**
        1. User opens the group chat interface
        2. User types and sends message
        3. System broadcasts message to all group members
        4. All group members receive notification
        5. Message appears in group chat with senders name and timestamp
    - **Extensions:**
        - 4a: some group members are offline

        1. System stores message for offlines members
        2. Offline members get message when reconnected
    - 2a: users want to mention specific members
        1. User types @ and members name
        2. System highlights mentioned member in message
        3. Mentioned member received special notification
    - 3a: admin sends urgent message
        1. System marks message as priority
        2. Members receive high priority notification
        3. Message appears highlighted in chat
  - **Special requirements:**
    - Support for up to 300 members in a single group chat
    - Messages must be ordered chronologically across all devices
    - Group admins can pin messages
  - **Technology and data variation list:**
    - Real time group messaging
    - Push notifications
    - Real time updated
  - **Frequency of occurrence:**
    - Often due to being used for event coordination and group communication
- Quick message buttons
  - **Primary Actor:**
    - passenger or driver
  - **Stakeholders and interests:**
    - Passenger: wants to communication quickly without typing on the go
    - Driver: wants to send status updates without typing while driving
    - Safety advocate: wants to decrease distractions while on the go
  - **Preconditions:**
    - User has active conversation with driver/passenger
    - User is logged into the app
  - **Success guarantee:**
    - Pre-written message sent with one tap
    - Recipient gets message immediately
    - No typing required

        - Message appears in conversation history
    - **Main success scenario:**
        1. User opens conversation with assigned driver/passenger
        2. User seed quick message buttons
        3. User taps a quick message button
        4. System sends pre-written message
        5. User received message with notification
    - **Extensions:**
        - 3a: user wants to add context to quick message
            1. User long presses quick message
            2. System opens text field
            3. User can edit message before sending
        - 2a: driver is in “driving mode”
            1. Quick message enlarged for easier tapping
            2. More important buttons are showed first
        - 5a: recipient has messages muted
            1. Message is still delivered but no sound/vibration
            2. Visual notification appears when app is opened
    - **Special requirements:**
        - Buttons must be large enough and easily accessible
        - Different quick messages for drivers and passengers
        - Voice to text alternative
    - **Technology and data variation list:**
        - Pre defined message templated
        - Text to speech for accessibility
        - Haptic feedback on button long press
    - **Frequency of occurrence:**
        - Very often due to being used to reduce typing during active pickups
- Messaging permissions
    - **Primary Actor:**
        - user (passenger, driver or admin)
    - **Stakeholders and interests:**
        - User: Wants control over who can send them messages and when
        - Group Admin: Wants to set default messaging rules for the group
        - Driver: Wants to ensure they can always contact assigned

passengers

- **Preconditions:**
  - User has an account and is logged into the app
  - User is a member of at least one group
- **Success guarantee:**
  - User's messaging permissions are saved and enforced
  - Only authorized users can send messages based on permissions
  - System respects user's privacy preferences
- **Main success scenario:**
  1. User navigates to messaging settings
  2. User selects who can message them
  3. System saves permission settings
  4. System enforces permissions for incoming messages
- **Extensions:**

  4a. Unauthorized user attempts to send message

    1. System blocks the message
    2. System notifies sender messaging is restricted

  2a. User wants different permissions for different groups

    1. User selects specific group
    2. User sets custom permissions for that group

  2b. Admin sets group-wide messaging policy

    1. Admin sets default permissions for all group members
    2. Individual members can override with stricter settings
- **Special requirements:**
  - Permission changes take effect immediately
  - Users cant block messages from assigned DD during active trips
  - Admins can override certain restrictions for emergencies
- **Technology and data variation list:**
  - Permission setting stored in user profile DB
  - Real time permission validation
  - Group level and user level hierarchies
- **Frequency of occurrence:**

        - Occasionally due to users set permissions once and modify when needed
- Messaging notifications
    - **Primary Actor:**
        - system (automated)
    - **Stakeholders and interests:**
        - Passenger: wants to be notified in proper time when driver sends a message
        - Driver: wants to know when passenger send message without having app open at all times
        - User: wants preference over notification style, sound, and frequency
    - **Preconditions:**
        - User has granted notification preferences to app
        - User has at least one active group membership or conversation
    - **Success guarantee:**
        - User gets timely notification for new messages
        - Notifications respect users notification preferences
        - User can go from notification to message smoothly
        - Notification count badge updates correctly
    - **Main success scenario:**
            1. Message sent to user
            2. System checks user notification preferences
            3. System sends push notification to user device
            4. User tap notification
            5. App opens to conversation
    - **Extensions:**
        - 2a: user has notifications muted for conversation
            1. System skips sending notification
            2. Message still shows in app with unread symbol
            3. Badge count still increases
        - 2b: user has “DND” enabled
            1. System queues notification silently
            2. Delivers notification when DND ends
            3. Badge count still updates
        - 4a: user is actively viewing conversation
            1. System shows in-app banner instead of push notification
            2. No sound plays

        3. Message appears immediately
    - 3a: multiple messages arrive in quick succession
        1. System groups notifications
        2. Shows “# new messages from ...”
    - 2c: message is from admin marked as urgent
        1. System override DND is critical alerts are enabled
        2. Sends high-priority notifications with sound
- **Special requirements:**
    - Notifications but arrive within 3 seconds of being sent
    - Users can customize notification sounds per conversation/group
    - Support for notification grouping/building for spam prevention
    - Badge count shows total unread messages
    - Must work on both iOS and android
- **Technology and data variation list:**
    - iOS push notification service
    - Android firebase cloud messaging
    - Local notifications for in app alerts
    - Background notification processing
- **Frequency of occurrence:**
    - Very often due to notifications being triggered when messages are sent

Thomas - Profiles/Vehicle/Ratings

- Driver bios/profile page
    - **Primary Actor:**
        - Driver
    - **Stakeholders and interests:**
        - Driver: Wants to add and update information on their profile
    - **Preconditions:**
        - An account has already been created by the Driver
    - **Success guarantee:**
        - Guide Users to their profile once they create their account and have them update it before continuing.
    - **Main success scenario:**
        - A Driver adds bio/information to their profile page
    - **Extensions:**
        - Driver only wants friends to be able to access their profile

            - Allow users to customize who can view their profile
        - A Drivers Profile is out of date
            - Prompt Users to update their profile after a certain amount of time
            - Automatically update certain profile information
    - **Special requirements:**
        - Updates to a User profile are published globally within 5 seconds of confirming the changes
    - **Technology and data variation list:**
        - Database entry for each User holds their Driver profile information
    - **Frequency of occurrence:**
        - Relatively frequently, but can be slightly delayed compared to other components
- Vehicle information
    - **Primary Actor:**
        - Passengers
    - **Stakeholders and interests:**
        - Passengers: Access the vehicle information so they know what car they can expect to be picked up from
        - Drivers: Fills out the information on the car they will be using for the drive
    - **Preconditions:**
        - A Drive is set and a driver/s is picked out from the Users within the group
    - **Success guarantee:**
        - The driver adds their vehicles information to the Drive
    - **Main success scenario:**
        - A Passenger who is waiting to get picked up accesses the Vehicle Information and sees that the driver car is a Toyota Corolla
    - **Extensions:**
        - The Driver never inputs their vehicle information
            - Their vehicle information from the last time they were the driver is put in, with a warning that it may be outdate
    - **Special requirements:**
        - Vehicle information is published globally within 5 seconds of confirming the information

    - **Technology and data variation list:**
        - Car Model Inputs
    - **Frequency of occurrence:**
        - Relatively frequently, but can be slightly delayed compared to other components
- Driver profile visible to assigned rider
    - **Primary Actor:**
        - Passenger
    - **Stakeholders and interests:**
        - Driver: Adds and updates information on their profile
        - Passenger: Passengers can use this page to see who they are being driven by
    - **Preconditions:**
        - A Driver is assigned by the group for a given Drive
        - That Driver has already created their profile page and added all relevant information
    - **Success guarantee:**
        - Users can access a Driver's Profile from their group
    - **Main success scenario:**
        - A passenger can click on one of the Drivers in their group and view their profile.
    - **Extensions:**
        - A Driver's profile is not accessible by Passenger due to view restrictions set by Driver
            - Passenger can appeal/ask for permission
        - A Driver's Profile is out of date
            - Prompt Users to update their profile after a certain amount of time
            - Automatically update certain profile information
    - **Special requirements:**
        - It takes at most 0.5 seconds to access a Driver's Profile once clicking on them from the group
    - **Technology and data variation list:**
        - Database entry for each User holds their profile information
    - **Frequency of occurrence:**
        - Relatively frequently, but can be slightly delayed compared to other components
- Two way ratings (rider → DD, DD → rider)

- **Primary Actor:**
  - Passenger, Driver
- **Stakeholders and interests:**
  - Driver: Wants to rate their passengers based on how they acted as a passenger during the drive and be rated positively by passengers
  - Passenger: Wants to rate their driver based on how they acted as a driver during the drive and be rated positively by the driver
- **Preconditions:**
  - The driver and passenger both have accounts set up
  - A drive has been completed
- **Success guarantee:**
  - A rating is given to either the driver by the passenger, the passenger by the driver, or both
  - The overall rating for each User is updated on their profile
- **Main success scenario:**
  - A drive is completed and a passenger wants to rate their driver highly. They rate the driver by going on their profile and clicking on the “Rate Me” button. They choose between 0 and 5 stars, and press submit. The Passenger receives a confirmation message and the Drivers overall rating is updated.
- **Extensions:**
  - A driver wants to rate a passenger,
    - Go through the same process of going onto the passengers profile, pressing ‘Rate Me’, and rating between 0 and 5 stars
  - A User wants to see both their passenger rating and their driver rating.
    - They can go onto their profile and automatically displayed at the top is their driver rating and passenger rating
- **Special requirements:**
  - Rating information is globally updated within 5 seconds of a new rating, for both driver and passenger ratings
- **Technology and data variation list:**
  - Database entry for each User holds their Passenger and Driver ratings
- **Frequency of occurrence:**

        - Relatively frequently, but can be slightly delayed compared to other components
- Reputation summary (avg rating)
    - **Primary Actor:**
        - Users
    - **Stakeholders and interests:**
        - User: They want to see what ratings they have been given by other Users
        - Other Users: Other Users want to see what the ratings are of someone within their group, or who is driving/being a passenger
    - **Preconditions:**
        - An account has been created by the User
        - A new rating has been given to a User for either Driver or Passenger ratings
    - **Success guarantee:**
        - The reputation summary automatically updates, displaying the average rating between both Driver and Passenger Ratings
    - **Main success scenario:**
        - A Passenger wants to see the reputation summary of their Driver before a Drive starts. They go to the Drivers profile and at the very top, the reputation summary is displayed. This is the average between the Driver and Passenger ratings. The User then exits out of the Drivers profile.
    - **Extensions:**
        - The User has not been a Driver or Passenger before
            - Their reputation summary is displayed as 'Incomplete'
        - The User has either not been a Driver or not been a Passenger before
            - Their reputation summary is displayed as 'Incomplete' and will only be shown once the user has been both a Driver and a Passenger
    - **Special requirements:**
        - Automatically update the Reputation summary globally within 5 seconds of a new rating being given, either for Driver or Passenger
    - **Technology and data variation list:**
        - Database entry for each User holds their reputation summary

rating
- **Frequency of occurrence:**
  - Relatively frequently, but can be slightly delayed compared to other components

Aryaan - GPS tracking + Privacy

- Driver GPS tracking during route
  - **Primary Actor:** Driver
  - **Stakeholders and interests:**
    - Driver: Wants accurate navigation and safe routing
    - Rider: Wants to see where the driver is in real time
    - Company/Admin: Wants reliable tracking for safety and records
  - **Preconditions:**
    - Driver is logged in and has accepted a ride request
    - Location permissions are granted
    - Internet access is available
  - **Success guarantee:**
    - Driver's location is captured and transmitted at proper intervals while the trip is active
  - **Main success scenario:**
    - The driver logs in and starts his dd shift making his location active to the riders
    - The rider requests the driver to pick them up at a certain location and the driver accepts
    - The app reads the GPS coordinates to the driver and sends location updates to the rider at the same time
  - **Extensions:**
    - 3a: GPS accuracy is poor
      - System reports last known location
      - System displays accuracy radius
      - System adjusts update interval
    - 4a: network connection lost
      - System stores updates locally
      - System sends updates when connection returns
      - User sees reconnecting stawtus
  - **Special requirements:**
    - Tracking only occurs during active trips
    - Updates must be near real-time

        - Must minimize battery drain
    - **Technology and data variation list:**
        - GPS or network-based location services
        - Update interval based on time or movement
        - Data includes latitude, longitude, timestamp, accuracy
    - **Frequency of occurrence:**
        - Very frequent, about every second to update live location
- Share driving location with assigned router
    - **Primary Actor:** Router/Dispatcher
    - **Stakeholders and interests:**
        - Router/Dispatcher: Needs live driver location to manage routes and monitor progress
        - Driver: Wants efficient routing support without distractions
        - Rider: Wants accurate ETAs based on real driver location
        - Company/Admin: Needs visibility for operations and safety
    - **Preconditions:**
        - Driver has accepted a ride and tracking is active
        - Router/Dispatcher is assigned to the trip
        - Internet connection is available
    - **Success guarantee:**
        - Router/Dispatcher can view the driver's real-time location during the active trip
    - **Main success scenario:**
        - Driver accepts a ride and GPS tracking begins
        - Router/Dispatcher opens the active trip dashboard
        - System verifies the router is authorized for the trip
        - System streams driver location updates to the router's dashboard
        - Router views driver position and route progress in real time
    - **Extensions:**
        - 2a: unauthorized access attempt
            - System denies access
            - System logs attempt
        - 3a: network issue
            - System shows last known location
            - System displays last update timestamp
    - **Special requirements:**
        - Access control (only assigned router can view)

        - Near real-time updates
        - Access activity should be logged
    - **Technology and data variation list:**
        - WebSockets or API polling for updates
        - Map integration (Google Maps, Mapbox, etc.)
        - Data includes latitude, longitude, timestamp, trip ID
    - **Frequency of occurrence:**
        - Occurs during every active trip when a router is monitoring it
- Rider live map view
    - **Primary Actor:** Rider
    - **Stakeholders and interests:**
        - Rider: Wants to see the driver's location and estimated arrival time
        - Driver: Wants rider informed to reduce calls/messages
        - Company/Admin: Wants transparency and fewer disputes
    - **Preconditions:**
        - Rider has requested a ride
        - A driver has accepted the request
        - Rider is logged in and has internet access
    - **Success guarantee:**
        - Rider can see the driver's real-time location and ETA during the active trip
    - **Main success scenario:**
        - Rider opens the app after booking a ride
        - System confirms a driver is assigned
        - App loads a map centered on the pickup location
        - Driver's location marker appears on the map
        - App updates the driver's position in real time
        - ETA and trip status are displayed to the rider
    - **Extensions:**
        - 2a: no driver assigned yet
            - System displays "Searching for driver"
            - System refreshes automatically
        - 4a: temporary signal loss
            - System shows last known driver location
            - System displays last updated time
    - **Special requirements:**
        - Location updates should appear smooth (no large jumps)

        - ETA must refresh dynamically
        - Driver location only visible during active trip (privacy rule)
    - **Technology and data variation list:**
        - Map provider integration (Google Maps, Mapbox, etc.)
        - Real-time updates via push or polling
        - Data includes latitude, longitude, timestamp, ETA, trip status
    - **Frequency of occurrence:**
        - Occurs during every active ride from booking until completion
- Arrival detection helper
    - **Primary Actor:**
        - System
    - **Stakeholders and interests:**
        - Rider: Wants accurate notification when driver arrives
        - Driver: Wants automatic arrival detection without extra steps
        - Company/Admin: Needs accurate arrival timestamps for records
    - **Preconditions:**
        - Trip is active
        - Pickup or drop-off location is set
        - Driver GPS tracking is active
    - **Success guarantee:**
        - System correctly detects arrival and updates trip status
    - **Main success scenario:**
        - System monitors driver GPS location
        - System compares driver location to pickup or drop-off location
        - Driver enters defined arrival radius
        - System confirms arrival after short dwell time
        - System updates trip status to “Arrived”
        - Rider receives arrival notification
        - Arrival timestamp is recorded
    - **Extensions:**
        - 3a: false arrival trigger
            - System checks dwell time
            - System checks driver speed
            - System confirms arrival only if conditions are met
        - 4a: wrong pickup location
            - Driver or rider adjusts pickup pin
            - System updates arrival radius

    - **Special requirements:**
        - Must reduce false arrival detections
        - Arrival radius should be configurable
        - Manual override option for driver if auto-detection fails
    - **Technology and data variation list:**
        - Geofencing via OS or server-side detection
        - Adjustable radius based on environment
        - Data includes driver coordinates, target coordinates, timestamp, speed
    - **Frequency of occurrence:**
        - Occurs once at pickup and once at drop-off for each trip
- Tracking privacy rules (hide location when not active)
    - Non functional requirement
    - The app must enforce privacy controls so the drivers live location is only available during the drivers assigned shift and only visible to the people that are riders